

ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group XX: [names and student IDs]

2026-07-15

Contents

Authorship and contribution statement	2
Abstract	2
Reproducible setup	2
Shared helper functions	3
1 Problem 1. Prediction design and leakage control	5
1.1 1A. Target and predictors	5
1.2 1B. Split and train-only preprocessing	6
1.3 1C. Training-data exploration	6
2 Problem 2. OLS, ridge, and lasso	14
2.1 2A. OLS baseline	14
2.2 2B. Ridge	14
2.3 2C. Lasso	14
2.4 2D	19
2.5 2E. Perturbation or stability check	19
3 Problem 3. Mathematical mechanisms	20
3.1 3A. Ridge and conditioning	20
3.2 3B. Lasso optimality and soft thresholding	20
3.3 3C. Connect algebra to evidence	20

4	Problem 4. Elastic net and a neural-feature bridge	20
4.1	4A. Research question and source map	20
4.2	4B. Elastic net on original predictors	20
4.3	4C. Fixed ReLU features	21
4.4	4D. Locked declaration and final holdout evaluation	21
4.5	4E. Deep-learning connection and limitations	21
5	Problem 5. Report quality and reproducibility	21
5.1	5A. Reproduction record	21
5.2	5B. Recommendation and limitations	21
5.3	5C. AI verification record	22
	Preflight and session information	22

Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in Group04_AI_Log.csv, and the group checked every AI-assisted item used in this report.

Student	ID	Main contributions	Verification performed
Nguyen Thi Yen Nhi	24125071	[Work]	[Check]
Nguyen Khang Thinh	24125104	[Work]	[Check]
Nguyen Van Tinh	24125106	[Work]	[Check]
Nguyen Le Quoc Huy	24125100	[Work]	[Check]

Abstract

Maximum 150 words: prediction problem, methods, main evidence, and recommendation.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr", "car")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
    paste(missing_packages, collapse = ", "))
}
```

```

}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)
library(car)

group_number <- as.integer(params$group_number)
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)

```

Shared helper functions

```

find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}

```

```

fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
  scale <- sqrt(colMeans(centered^2))
  keep <- is.finite(scale) & scale > tol
  if (!any(keep)) stop("No nonconstant training predictor remains.")
  list(
    center = center[keep],
    scale = scale[keep],
    keep = keep,
    dropped = colnames(x)[!keep]
  )
}

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),

```

```

    MAE = mean(abs(y - prediction)))
  }

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}

```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```

config <- list(
  file = "prostate.csv",
  response = "lpsa",
  excluded = "lpsa"
)
data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}

```

Define the prediction task and justify every exclusion.

1.2 1B. Split and train-only preprocessing

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
test_id <- split$test

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test <- y_raw[test_id]

foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
sum(is.na(analysis_data))
```

```
## [1] 0
```

```
save(x_train, y_train, x_test, y_test, analysis_data, train_id, test_id, foldid, file = "data/
```

Report seeds, dimensions, scaling, folds, and leakage safeguards.

1.3 1C. Training-data exploration

```
# Use analysis_data[train_id, ] only. Add focused plots and summaries.
train_df <- analysis_data[train_id, ]

show(train_df)
```

```
##      lpsa      lcavol lweight age      lbph svi      lcp gleason pgg45
## 1 -0.43078 -0.5798185 2.7695 50 -1.386294 0 -1.38629      6      0
## 2 -0.16252 -0.9942523 3.3196 58 -1.386294 0 -1.38629      6      0
## 3 -0.16252 -0.5108256 2.6912 74 -1.386294 0 -1.38629      7     20
## 4 -0.16252 -1.2039728 3.2828 58 -1.386294 0 -1.38629      6      0
## 6  0.76547 -1.0498221 3.2288 50 -1.386294 0 -1.38629      6      0
## 7  0.76547  0.7371641 3.4735 64  0.615186 0 -1.38629      6      0
## 8  0.85442  0.6931472 3.5395 58  1.536867 0 -1.38629      6      0
## 9  1.04732 -0.7765288 3.5395 47 -1.386294 0 -1.38629      6      0
## 10 1.04732  0.2231436 3.2445 63 -1.386294 0 -1.38629      6      0
## 11 1.26695  0.2546422 3.6041 65 -1.386294 0 -1.38629      6      0
## 12 1.26695 -1.3470736 3.5987 63  1.266948 0 -1.38629      6      0
```

## 14	1.34807	1.4770487	2.9982	67	-1.386294	0	-1.38629	7	5
## 17	1.47018	-0.4155154	3.5160	70	1.244155	0	-0.59784	7	30
## 18	1.49290	2.2884862	3.6494	66	-1.386294	0	0.37156	6	0
## 20	1.59939	0.1823216	3.8254	70	1.658228	0	-1.38629	6	0
## 21	1.63900	1.1474025	3.4194	59	-1.386294	0	-1.38629	6	0
## 23	1.69562	-0.5447272	3.3759	59	-0.798508	0	-1.38629	6	0
## 24	1.71380	1.7817091	3.4516	63	0.438255	0	1.17865	7	60
## 25	1.73166	0.3852624	3.6674	69	1.599388	0	-1.38629	6	0
## 26	1.76644	1.4469190	3.1246	68	0.300105	0	-1.38629	6	0
## 27	1.80006	0.5128236	3.7197	65	-1.386294	0	-0.79851	7	70
## 28	1.81645	-0.4004776	3.8660	67	1.816452	0	-1.38629	7	20
## 29	1.84845	1.0402767	3.1290	67	0.223144	0	0.04879	7	80
## 30	1.89462	2.4096442	3.3759	65	-1.386294	0	1.61939	6	0
## 31	1.92425	0.2851789	4.0902	65	1.962908	0	-0.79851	6	0
## 32	2.00821	0.1823216	6.1076	65	1.704748	0	-1.38629	6	0
## 33	2.00821	1.2753628	3.0374	71	1.266948	0	-1.38629	6	0
## 35	2.04769	-0.0100503	3.2169	63	-1.386294	0	-0.79851	6	0
## 36	2.08567	1.3083328	4.1198	64	2.171337	0	-1.38629	7	5
## 38	2.19165	0.4574248	2.3749	64	-1.386294	0	-1.38629	7	15
## 39	2.21375	2.6609586	4.0851	68	1.373716	1	1.83258	7	35
## 40	2.27727	0.7975072	3.0131	56	0.936093	0	-0.16252	7	5
## 41	2.29757	0.6205765	3.1420	60	-1.386294	0	-1.38629	9	80
## 42	2.30757	1.4422020	3.6826	68	-1.386294	0	-1.38629	7	10
## 43	2.32728	0.5822156	3.8660	62	1.713798	0	-0.43078	6	0
## 44	2.37491	1.7715568	3.8969	61	-1.386294	0	0.81093	7	6
## 45	2.52172	1.4861397	3.4095	66	1.749200	0	-0.43078	7	20
## 46	2.55334	1.6639261	3.3928	61	0.615186	0	-1.38629	7	15
## 47	2.56879	2.7278528	3.9954	79	1.879465	1	2.65676	9	100
## 48	2.56879	1.1631508	4.0351	68	1.713798	0	-0.43078	7	40
## 49	2.59152	1.7457155	3.4980	43	-1.386294	0	-1.38629	6	0
## 50	2.59152	1.2208299	3.5681	70	1.373716	0	-0.79851	6	0
## 51	2.65676	1.0919233	3.9936	68	-1.386294	0	-1.38629	7	50
## 52	2.67759	1.6601310	4.2348	64	2.073172	0	-1.38629	6	0
## 53	2.68444	0.5128236	3.6336	64	1.492904	0	0.04879	7	70
## 55	2.70471	3.1535904	3.5160	59	-1.386294	0	-1.38629	7	5
## 56	2.71800	1.2669476	4.2801	66	2.122262	0	-1.38629	7	15
## 57	2.78809	0.9745596	2.8651	47	-1.386294	0	0.50078	7	4
## 60	2.81241	1.0612565	3.8512	61	1.294727	0	-1.38629	7	40
## 61	2.84200	0.4574248	4.5245	73	2.326302	0	-1.38629	6	0
## 63	2.85359	2.7757088	3.5249	72	-1.386294	0	1.55814	9	95
## 64	2.88200	2.0347056	3.9170	66	2.008214	1	2.11021	7	60
## 68	2.96269	2.1983351	4.0509	72	2.307573	0	-0.43078	7	10
## 69	2.96269	-0.4462871	4.4085	69	-1.386294	0	-1.38629	6	0
## 71	3.01308	1.8640801	3.5932	60	-1.386294	1	1.32176	7	60
## 73	3.05636	1.2149127	3.8254	69	-1.386294	1	0.22314	7	20
## 74	3.07501	1.8389611	3.2367	60	0.438255	1	1.17865	9	90
## 75	3.27526	2.9992262	3.8491	69	-1.386294	1	1.90954	7	20
## 76	3.33755	3.1411305	3.2638	68	-0.051293	1	2.42037	7	50

```
## 77 3.39283 2.0108950 4.4338 72 2.122262 0 0.50078 7 60
## 78 3.43560 2.5376572 4.3548 78 2.326302 0 -1.38629 7 10
## 79 3.45789 2.6483002 3.5821 69 -1.386294 1 2.58400 7 70
## 80 3.51304 2.7794402 3.8232 63 -1.386294 0 0.37156 7 50
## 81 3.51601 1.4678743 3.0704 66 0.559616 0 0.22314 7 40
## 82 3.53076 2.5136561 3.4735 57 0.438255 0 2.32728 7 60
## 83 3.56530 2.6130067 3.8888 77 -0.527633 1 0.55962 7 30
## 84 3.57094 2.6775910 3.8384 65 1.115142 0 1.74920 9 70
## 85 3.58768 1.5623463 3.7099 60 1.695616 0 0.81093 7 30
## 86 3.63099 3.3028493 3.5190 64 -1.386294 1 2.32728 7 60
## 88 3.71235 1.7316555 3.3690 62 -1.386294 1 0.30010 7 30
## 90 3.99360 1.5623463 3.6951 76 0.936093 1 0.81093 7 75
## 91 4.02981 3.2464910 4.1018 68 -1.386294 0 -1.38629 6 0
## 92 4.12955 2.5329028 3.6776 61 1.348073 1 -1.38629 7 15
## 94 4.68444 3.8210036 3.8969 44 -1.386294 1 2.16905 7 40
## 95 5.14312 2.9074474 3.3962 52 -1.386294 1 2.46385 7 10
## 96 5.47751 2.8825636 3.7739 68 1.558145 1 1.55814 7 80
## 97 5.58293 3.4719665 3.9750 68 0.438255 1 2.90417 7 20
```

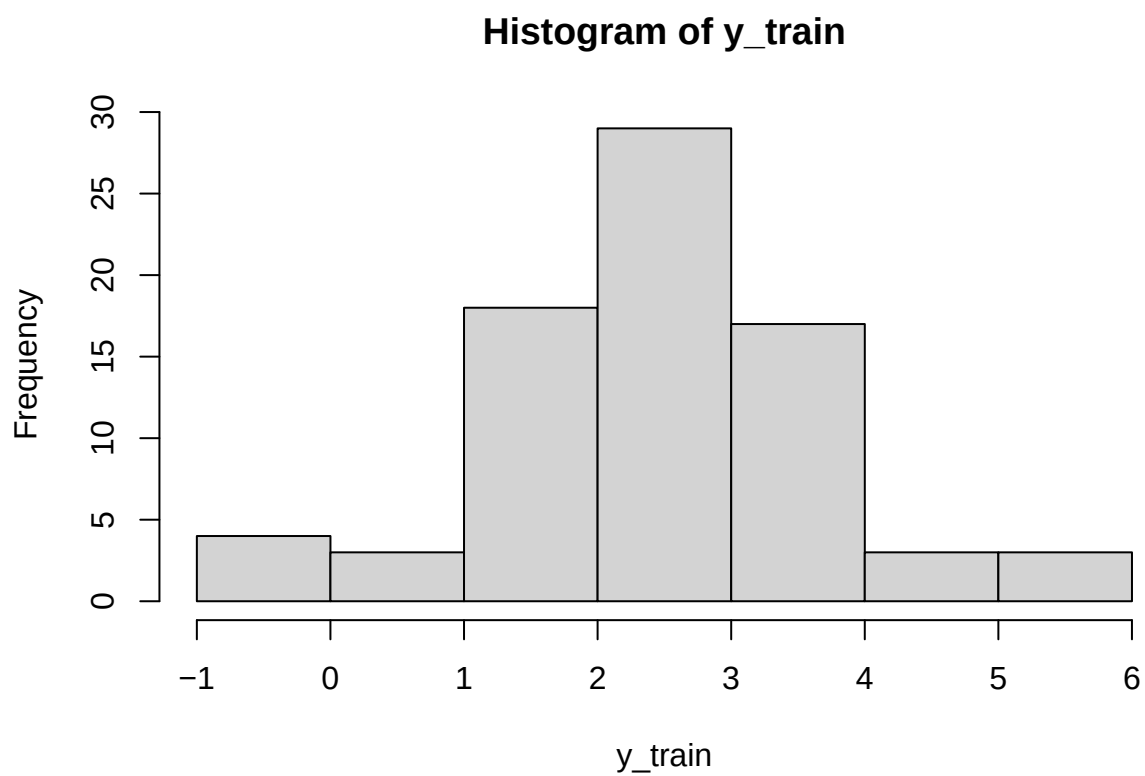
```
# Pairwise correlation among predictors
cor_matrix <- cor(
  train_df[, predictors],
  use = "complete.obs"
)

knitr::kable(
  round(cor_matrix, 2),
  caption = "Pairwise correlation matrix among predictors"
)
```

Table 1: Pairwise correlation matrix among predictors

	lcavol	lweight	age	lbph	svi	lcp	gleason	pgg45
lcavol	1.00	0.17	0.17	0.01	0.55	0.68	0.44	0.41
lweight	0.17	1.00	0.31	0.44	0.07	0.03	-0.06	-0.01
age	0.17	0.31	1.00	0.38	0.11	0.08	0.19	0.24
lbph	0.01	0.44	0.38	1.00	-0.07	-0.04	0.05	0.06
svi	0.55	0.07	0.11	-0.07	1.00	0.69	0.33	0.43
lcp	0.68	0.03	0.08	-0.04	0.69	1.00	0.49	0.61
gleason	0.44	-0.06	0.19	0.05	0.33	0.49	1.00	0.78
pgg45	0.41	-0.01	0.24	0.06	0.43	0.61	0.78	1.00

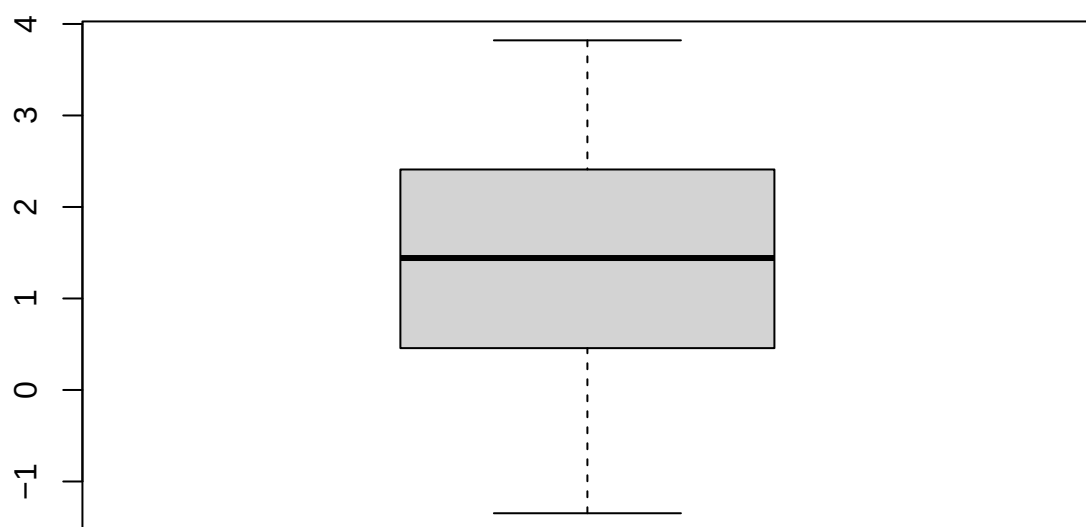
```
# Response Distribution
hist(y_train)
```

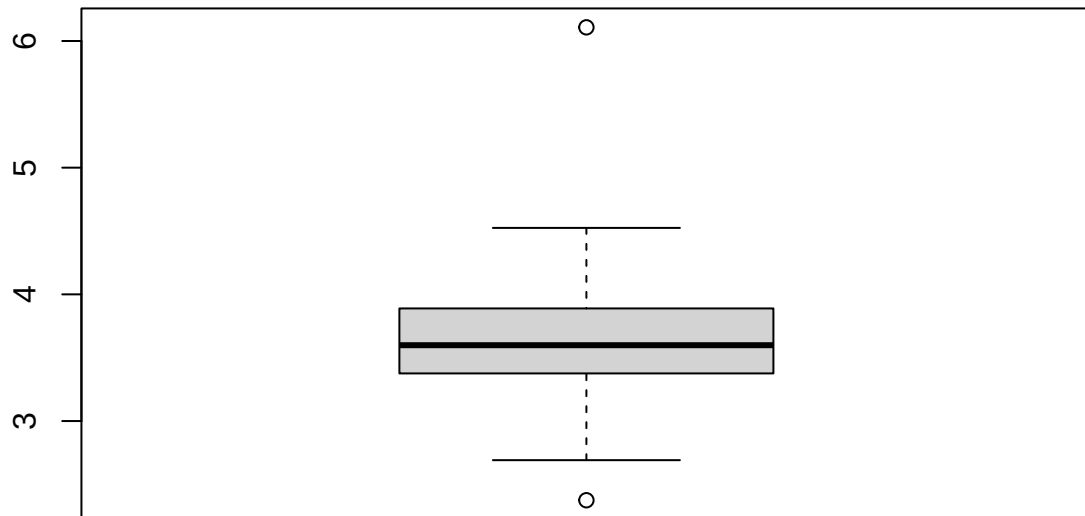
```
summary(y_train)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.4308  1.7664   2.5688   2.4757  3.2753   5.5829
```

```
# Predictor Distribution
boxplot(train_df$lcavol)
```



```
boxplot(train_df$lweight)
```



```
#.....

# Correlation between predictors and response
cor_response <- cor(
  train_df[, predictors],
  train_df$lpsa
)

knitr::kable(
  round(cor_response, 2),
  caption = "Correlation between predictors and lpsa"
)
```

Table 2: Correlation between predictors and lpsa

lcavol	0.77
lweight	0.30
age	0.14
lbph	0.14
svi	0.57
lcp	0.59

gleason	0.39
pgg45	0.41

```
# Check multicollinearity using VIF
library(car)

vif_model <- lm(
  lpsa ~ .,
  data = train_df[, c("lpsa", predictors)]
)

vif_values <- vif(vif_model)

knitr::kable(
  round(vif_values, 2),
  caption = "Variance Inflation Factor (VIF)"
)
```

Table 3: Variance Inflation Factor (VIF)

	x
lcavol	2.18
lweight	1.38
age	1.32
lbph	1.40
svi	1.97
lcp	3.29
gleason	2.79
pgg45	3.35

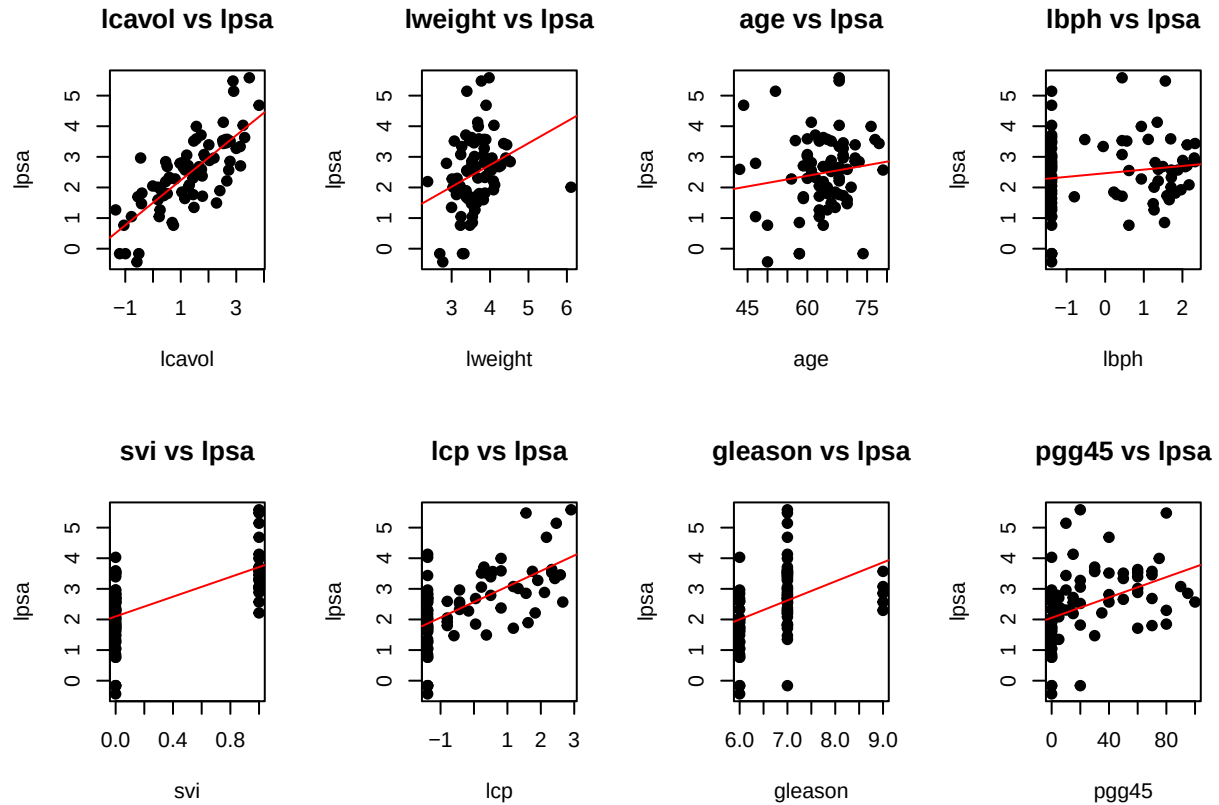
```
# Predictor-response plots
par(mfrow = c(2,4))

for (var in predictors) {
  plot(
    train_df[[var]],
    train_df$lpsa,
    xlab = var,
    ylab = "lpsa",
    main = paste(var, "vs lpsa"),
    pch = 19
  )

  abline(
    lm(train_df$lpsa ~ train_df[[var]]),

```

```
col = "red"
)
}
```



```
par(mfrow = c(1,1))
```

Interpret multicollinearity, one anomaly, and the question regularization will address.

1. Target distribution -> *lpsa* có skew/outlier không?
2. Predictor distribution -> feature có vấn đề không?
3. Predictor-response correlation -> feature nào có liên hệ với Y?
4. Predictor-predictor correlation -> feature nào có thông tin trùng nhau?
5. VIF -> multicollinearity có nghiêm trọng không cho từng predictor (so với predictor còn lại?)?

2 Problem 2. OLS, ridge, and lasso

2.1 2A. OLS baseline

```
# Fit OLS on x_train and y_train. Do not access y_test here.
```

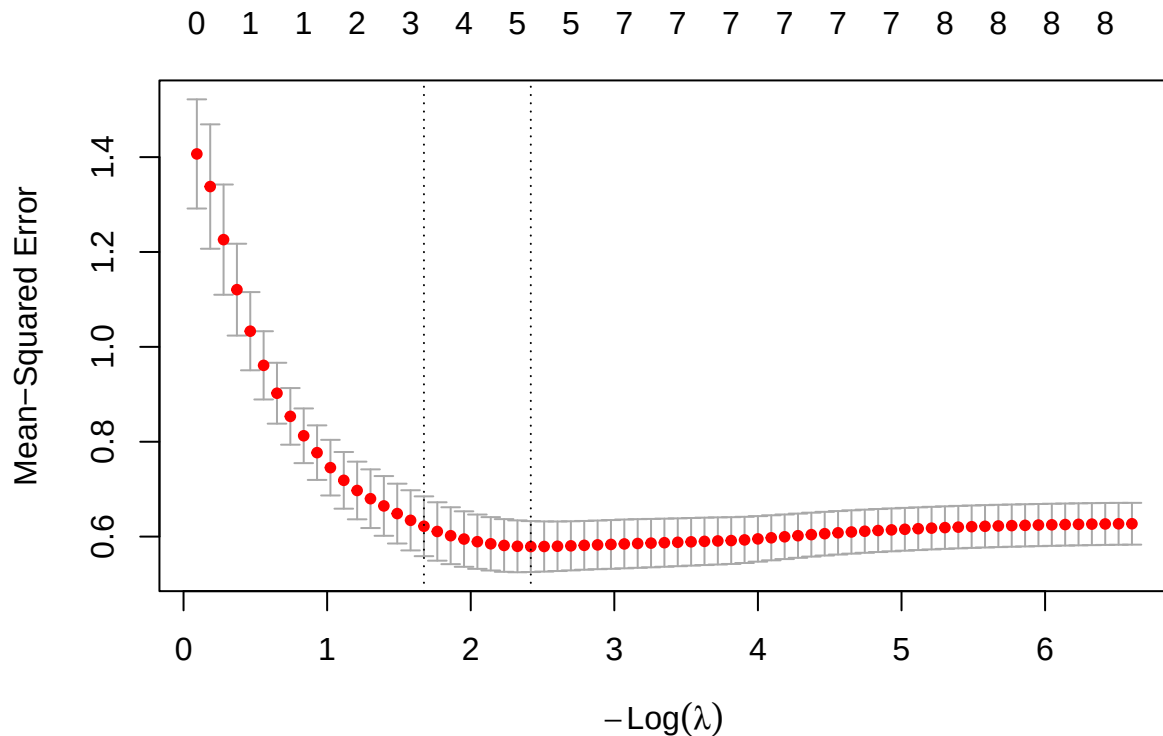
2.2 2B. Ridge

```
# Use fit_cv_glmnet(..., alpha = 0, foldid = foldid).  
# Report lambda.min, lambda.1se, paths, coefficients, and CV evidence.
```

2.3 2C. Lasso

2.3.1 Model fitting and cross-validation

```
lasso_cv <- fit_cv_glmnet(  
  x = x_train,  
  y = y_train,  
  alpha = 1,  
  foldid = foldid  
)  
plot(lasso_cv)
```



Based on the 5-fold cross-validation curve generated for the Lasso regression model ($\alpha = 1$), we observe the relationship between the tuning parameter λ and the Mean-Squared Error (MSE). The top axis indicates the number of active (non-zero) predictors retained in the model at each level of penalization.

- The Cross-Validation Curve: The red dots represent the average CV-MSE across the 5 folds for each λ value, with the gray error bars denoting one standard error above and below the mean. As $-\log(\lambda)$ increases (moving from left to right, meaning the penalty weakens), the MSE initially drops, reaches a minimum, and then slightly stabilizes as the model approaches the unpenalized OLS limit.
- Optimal λ ($\lambda_{\min}$): The right vertical dashed line marks `lambda.min`, the value that strictly minimizes the cross-validation MSE. At this optimal point for prediction accuracy ($-\log(\lambda) \approx 2.4$), the Lasso penalty retains 5 non-zero predictors*.
- Parsimonious λ (λ_{1se}): The left vertical dashed line marks `lambda.1se`, selected by the one-standard-error rule. This value applies a stronger penalty ($-\log(\lambda) \approx 1.67$) to produce a sparser model. It retains only 3 non-zero predictors while keeping the error within one standard deviation of the absolute minimum MSE, offering a robust trade-off between predictive power and model simplicity.

2.3.2 Analysis of $\lambda_{\min}$

```
lambda_min <- lasso_cv$lambda.min
coef(lasso_cv, s="lambda.min")

## 9 x 1 sparse Matrix of class "dgCMatrix"
##           lambda.min
## (Intercept) 2.47569468
## lcavol      0.67611312
## lweight     0.12774355
## age         .
## lbph        0.03085712
## svi         0.20454921
## lcp         .
## gleason     .
## pgg45       0.03234366

count_nonzero(lasso_cv, s="lambda.min")

## [1] 5
```

The optimal tuning parameter selected by five-fold cross-validation is $\lambda_{\min} = 0.0891$. This value minimizes the cross-validation Mean Squared Error (CV-MSE), providing the best predictive performance among all candidate penalty values.

At this level of regularization, the Lasso model retains five non-zero predictors: lcavol, lweight, lbph, svi, and pgg45. The remaining predictors (age, lcp, and gleason) are shrunk exactly to zero, demonstrating Lasso's ability to perform automatic variable selection while maintaining predictive accuracy.

The retained predictors suggest that tumor volume (lcavol) is the most influential variable, having the largest estimated coefficient (0.6761). Variables related to body weight (lweight), benign prostatic hyperplasia (lbph), seminal vesicle invasion (svi), and the percentage of Gleason grades 4 or 5 (pgg45) also contribute to predicting PSA levels, although with relatively smaller effects.

Overall, $\lambda_{\min}$ produces the model with the lowest cross-validation error while preserving a moderate number of predictors, offering a balance between prediction accuracy and model complexity.

```
cv_mse_min <- lasso_cv$cvm[
  lasso_cv$lambda==
  lasso_cv$lambda.min
]
cv_mse_min
```

```
## [1] 0.5789934
```


The minimum five-fold cross-validation Mean Squared Error (CV-MSE) achieved by the Lasso model is **0.579**. This value corresponds to $\lambda_{\min} = 0.0891$ and represents the estimated prediction error obtained during model tuning using the training data.

A lower CV-MSE indicates better expected predictive performance on unseen data. Since $\lambda_{\min}$ is selected to minimize this metric, it is the tuning parameter that prioritizes prediction accuracy over model simplicity.

2.3.3 Analysis of λ_{1se}

```
lambda_1se <- lasso_cv$lambda.1se
coef(lasso_cv, s="lambda.1se")

## 9 x 1 sparse Matrix of class "dgCMatrix"
##          lambda.1se
## (Intercept) 2.47569468
## lcavol      0.63661195
## lweight     0.05343261
## age         .
## lbph         .
## svi         0.14411468
## lcp         .
## gleason     .
## pgg45       .

count_nonzero(lasso_cv, s="lambda.1se")
```

```
## [1] 3
```

The one-standard-error rule selects $\lambda_{1se} = \text{rround}(\text{lambda}_{1se}, 4)$, which applies a stronger regularization penalty than $\lambda_{\min}$. The corresponding five-fold cross-validation Mean Squared Error (CV-MSE) is $\text{round}(\text{cv_mse_1se}, 4)$, remaining within one standard error of the minimum while producing a simpler model.

Under λ_{1se} , the Lasso model retains only three non-zero predictors (lcavol, lweight, and svi). Compared with $\lambda_{\min}$, the variables lbph and pgg45 are removed, resulting in a more parsimonious and interpretable model with only a small loss in predictive performance.

```
cv_mse_1se <- lasso_cv$cvm[
  lasso_cv$lambda ==
  lasso_cv$lambda.1se
]
cv_mse_1se
```

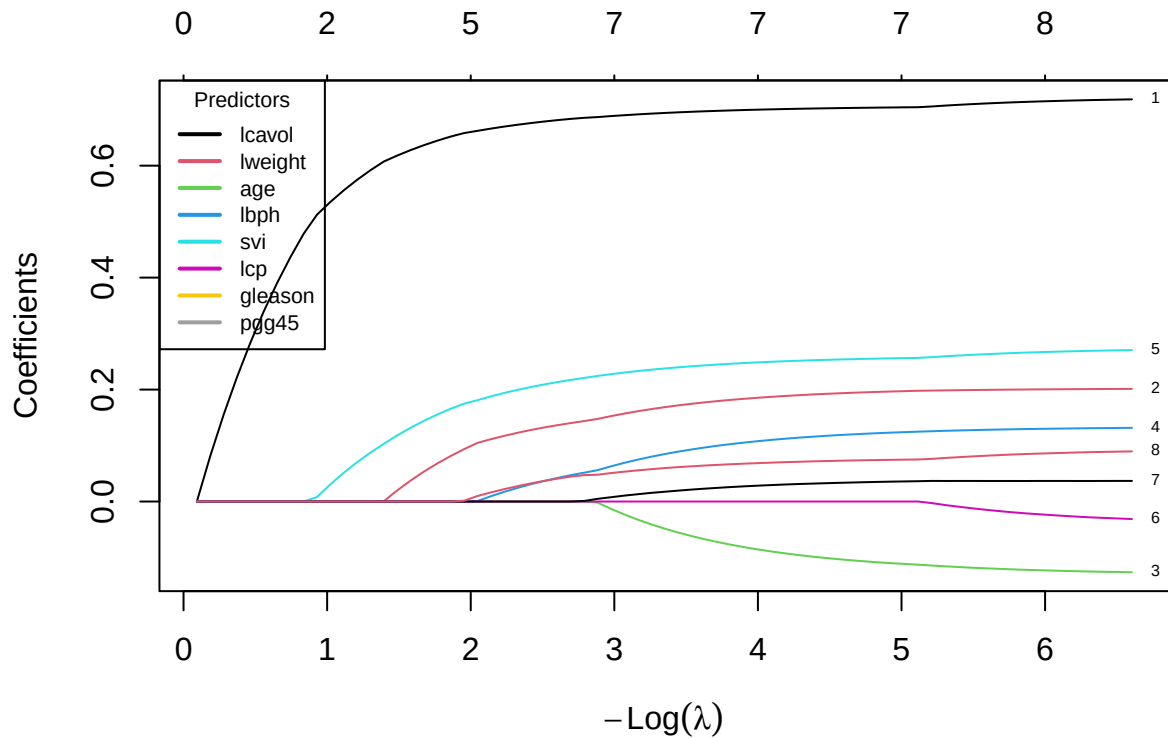
```
## [1] 0.6217887
```

The one-standard-error rule selects $\lambda_{1se} = \text{rround}(\text{lambda}_{1se}, 4)$, resulting in a five-fold cross-validation Mean Squared Error (CV-MSE) of $\text{rround}(\text{cv_mse}_{1se}, 4)$. Although this error is slightly higher than that of $\lambda_{\min}$, it remains within one standard error of the minimum while providing a more conservative model.

Under λ_{1se} , the Lasso model retains only three non-zero predictors (lcavol, lweight, and svi). Compared with $\lambda_{\min}$, the predictors lbph and pgg45 are removed, resulting in a simpler and more interpretable model with only a modest reduction in predictive performance.

2.3.4 Coefficient Path

```
lasso_fit <- glmnet(  
  x_train,  
  y_train,  
  alpha=1,  
  standardize=FALSE,  
  
)  
  
plot(lasso_fit, xvar = "lambda", label = TRUE)  
  
legend("topleft",  
  legend = rownames(lasso_fit$beta),  
  col = 1:nrow(lasso_fit$beta),  
  lty = 1,  
  cex = 0.7,  
  lwd = 2,  
  title = "Predictors")
```



Overall, the coefficient path shows that as $-\log(\lambda)$ increases (equivalently, as λ decreases), the regularization penalty becomes weaker, allowing more predictors to enter the model and their coefficients to increase in magnitude. Under strong regularization, many coefficients are shrunk exactly to zero, while weaker regularization gradually recovers the full model.

Among the predictors, **lcavol** enters the model first and consistently has the largest coefficient, indicating that it is the most influential predictor of **lpsa**. **svi** and **lweight** also become active relatively early and maintain positive coefficients throughout the path. In contrast, **age**, **lbph**, **lcp**, **gleason**, and **pgg45** enter only when the penalty becomes sufficiently small, suggesting that their contributions are comparatively weaker and less stable.

2.4 2D

```
# Build a comparison from training/CV quantities only.
```

Core model nominated before holdout evaluation: [method and reason]

2.5 2E. Perturbation or stability check

Prediction before running the check: [prediction]

```
# Implement one allowed training-only stability check.
```

Observed result and explanation: [result]

3 Problem 3. Mathematical mechanisms

3.1 3A. Ridge and conditioning

Write the ridge derivation. Define dimensions and explain uniqueness.

```
# Use x_train and the fitted ridge lambda.min.  
# safe_condition_numbers(x_train, lambda = ...)
```

3.2 3B. Lasso optimality and soft thresholding

Derive the zero/nonzero optimality conditions and the orthonormal-design formula.

3.3 3C. Connect algebra to evidence

Connect one specific numerical result to Problem 3A or 3B.

4 Problem 4. Elastic net and a neural-feature bridge

4.1 4A. Research question and source map

Research direction: [weight decay / sparse weights / pruning / dropout]

Claim	Exact primary source and locator	What it establishes	What it does not establish
[Claim]	[Citation, section/page/equation]	[Support]	[Boundary]

4.2 4B. Elastic net on original predictors

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)  
  
# Fit one cv.glmnet object per alpha with the same foldid.  
# Select alpha and lambda using training CV only.
```

4.3 4C. Fixed ReLU features

```
relu <- function(z) pmax(z, 0)
M <- 200L

# TODO:
# 1. Generate A and bias once from seeds$features using the stated distributions.
# 2. Apply the same A and bias to x_train and x_test.
# 3. Fit a scaler to H_train only and apply it to H_test.
# 4. Fit ridge, lasso, and elastic net on H_train with the shared foldid.
```

State which weights are fixed, which are learned, and how active features are counted.

4.4 4D. Locked declaration and final holdout evaluation

Locked before viewing y_test: [preprocessing, candidate specifications, selected tuning values, feature seed, nominated deployment model, metrics]

```
# This must be the first analysis chunk that uses y_test for evaluation.
# Generate all fixed candidate predictions and one RMSE/MAE table here.
```

State whether the holdout evidence supports the predeclared choice. Do not retune.

4.5 4E. Deep-learning connection and limitations

Distinguish penalty, weight decay, output-weight sparsity, pruning, dropout, and a trained deep network. Cite the sources from 4A and discuss at least two limitations.

5 Problem 5. Report quality and reproducibility

5.1 5A. Reproduction record

```
1. Open a clean R session.
2. [Exact commands and folder layout.]
3. Render GroupXX_ProjectQuiz1.Rmd.
```

5.2 5B. Recommendation and limitations

Give the final recommendation, distinguish prediction from interpretation, and state limitations.

5.3 5C. AI verification record

AI-assisted item	How it was checked	Evidence	Modification or rejection
[Item]	[Method]	[Test/source/output]	[Decision]

Preflight and session information

```
# Add every object required to reproduce the final table and figures.
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"
)
missing_objects <- setdiff(required_objects, ls())
if (length(missing_objects)) {
  stop(
    "Missing required objects: ",
    paste(missing_objects, collapse = ", ")
  )
}
stopifnot(length(intersect(train_id, test_id)) == 0L)
```

```
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Vietnamese_Vietnam.utf8  LC_CTYPE=Vietnamese_Vietnam.utf8
## [3] LC_MONETARY=Vietnamese_Vietnam.utf8 LC_NUMERIC=C
## [5] LC_TIME=Vietnamese_Vietnam.utf8
##
## time zone: Asia/Saigon
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
##  [1] car_3.1-5      carData_3.0-6  knitr_1.51     broom_1.0.13
##  [5] glmnet_5.0     Matrix_1.7-5   lubridate_1.9.5 forcats_1.0.1
```

```

## [9] stringr_1.6.0    dplyr_1.2.1      purrr_1.2.2      readr_2.2.0
## [13] tidyr_1.3.2      tibble_3.3.1     ggplot2_4.0.3    tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] generics_0.1.4    shape_1.4.6.1     stringi_1.8.7     lattice_0.22-9
## [5] hms_1.1.4         digest_0.6.39     magrittr_2.0.5    timechange_0.4.0
## [9] evaluate_1.0.5    grid_4.6.1        RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0     foreach_1.5.2     backports_1.5.1   Formula_1.2-5
## [17] survival_3.8-6    scales_1.4.0      codetools_0.2-20  abind_1.4-8
## [21] cli_3.6.6         rlang_1.3.0       splines_4.6.1     withr_3.0.3
## [25] yaml_2.3.12       otel_0.2.0        tools_4.6.1       tzdb_0.5.0
## [29] vctrs_0.7.3       R6_2.6.1          lifecycle_1.0.5   pkgconfig_2.0.3
## [33] pillar_1.11.1     gtable_0.3.6      glue_1.8.1        Rcpp_1.1.2
## [37] xfun_0.60         tidyselect_1.2.1  rstudioapi_0.19.0 farver_2.1.2
## [41] htmltools_0.5.9   rmarkdown_2.31    compiler_4.6.1    S7_0.2.2

```